Assignment 2 — IMDB Sentiment Classification with CNNs

DATA-MSML 612 — Deep Learning
University of Maryland

Shrikanth Vilvadrinath

March 6, 2026

Contents

1 Introduction **2**

2 Dataset **2**

3 Models and Architecture **3**
3.1 Logistic Regression (Baseline) 3
3.2 Feed-Forward Neural Network (Baseline) 3
3.3 Convolutional Neural Network (CNN) 3

4 Results **4**

5 Analysis **4**
5.1 Model Performance Hierarchy 4
5.2 FFNN vs. CNN — Why the Simpler Model Wins Here 4
5.3 Logistic Regression as a Strong Baseline 5
5.4 Effect of Stopword Removal (Task 6) 5

6 Training Curves **6**

7 Conclusion **8**

1 Introduction

This report presents a comparative study of three model families for binary sentiment classification on the IMDB movie-review dataset: Logistic Regression (LR), a Feed-Forward Neural Network (FFNN), and a Convolutional Neural Network (CNN). We additionally investigate the effect of stopword removal on CNN performance. The target accuracy threshold throughout is **¿85%**.

The results reveal a nuanced performance landscape: the FFNN slightly outperforms the CNN in test accuracy (86.50% vs. 85.57%), which is counter-intuitive given the CNN's richer inductive bias. Both neural models exhibit significant overfitting, which the analysis examines in depth. All three primary models clear the 85% threshold.

Framework: TensorFlow 2.10 / Keras. **Seed:** 42 (fully reproducible).

2 Dataset

The IMDB dataset [1] contains **50,000 movie reviews** split evenly as 25,000 for training and 25,000 for testing. Labels are binary (1 = positive, 0 = negative). The dataset is perfectly balanced: 12,500 positive and 12,500 negative examples in each split.

> *Note on the assignment description:* The assignment text states "50,000 reviews for training and the same amount again for testing." The actual Keras IMDB dataset provides **25,000 training and 25,000 test reviews (50,000 total)**. The numbers used here reflect the actual dataset.

Preprocessing

- Vocabulary limited to the top **5,000** most frequent words (`NUM_WORDS=5000`; rarer words replaced with the <`UNK`> token).
- Reviews padded/truncated to **500 tokens** (`MAX_LEN=500`) using pre-padding (zeros prepended to short reviews; long reviews truncated from the left).
- Raw review length statistics: min = 7, max = 2,494, mean = 234.8, median = 176.0.

Sample Decoded Review

To illustrate the preprocessing pipeline, training review 0 (Positive label) decodes as:

> *"this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert redford's is an amazing actor. . . "*

Reviews are stored as integer word-rank sequences and decoded back via the IMDB word index. The model sees integer tokens, not raw text.

3 Models and Architecture

3.1 Logistic Regression (Baseline)

Input: Multi-hot binary bag-of-words (BoW) vectors of shape (25,000, 5,000). Position i is 1 if word-ID i appears in the review, 0 otherwise.

Using raw padded integer sequences as LR input is semantically incorrect: word-rank 42 would be treated as twice as important as word-rank 21, which is meaningless. Multi-hot BoW is the standard representation for linear text classification.

Listing 1: Bag-of-Words encoding

```
def to_multi_hot(sequences, num_words):
    result = np.zeros((len(sequences), num_words), dtype=np.float32)
    for i, seq in enumerate(sequences):
        indices = seq[seq > 0]              # skip padding
        indices = indices[indices < num_words]
        result[i, indices] = 1.0
    return result
```

3.2 Feed-Forward Neural Network (Baseline)

Listing 2: FFNN architecture

```
Embedding(5000, 32, input_length=500)  ->  (batch, 500, 32)
Flatten()                              ->  (batch, 16000)
Dense(250, activation='relu')          ->  (batch, 250)
Dense(1,   activation='sigmoid')       ->  (batch, 1)
# Total parameters: 4,160,501
```

The FFNN uses a *learned* Embedding layer, giving it representation-learning capability. This is standard practice for neural text classification and provides a meaningful comparison point before adding convolutional layers. The model learns distributed word representations jointly with the classification head.

3.3 Convolutional Neural Network (CNN)

Listing 3: CNN architecture with dimension walkthrough

```
Embedding(5000, 32, input_length=500)          -> (batch, 500, 32)
Conv1D(32, kernel_size=7, activation='relu')   -> (batch, 494, 32)
    [500-7+1=494]
MaxPooling1D(pool_size=5)                      -> (batch,  98, 32) [floor
    (494/5)=98]
Flatten()                                      -> (batch, 3136)    [98*32=3136]
Dense(250, activation='relu')                  -> (batch, 250)
Dense(1,   activation='sigmoid')               -> (batch, 1)
# Total trainable parameters: 951,701
```

Architecture rationale:

- `kernel_size=7`: Captures 7-word n-gram patterns. A window of 7 is large enough to cover sentiment negation phrases such as *"not at all good"* or *"could not have been better"*.
- `pool_size=5`: Aggressive subsampling extracts the most salient feature activations, reducing the temporal dimension 494 → 98.
- `Dense(250)`: Standard-capacity classification head; $98 \times 32 = 3{,}136$ flattened inputs provide sufficient signal.
- `sigmoid` output with `binary_crossentropy` loss for binary classification.

Compile: `optimizer='adam'`, `loss='binary_crossentropy'`, `metrics=['accuracy']`.
Training: 10 epochs, `batch_size=128`, `validation_split=0.2` (test set evaluated only once after training to avoid data leakage).

4 Results

All models trained with `SEED=42` (`random`, `numpy`, and `tensorflow` seeds all set). Table 1 summarises the outcomes.

Table 1: Model accuracy results on IMDB dataset (SEED=42)

Model	**Train Acc**	**Test Acc**	**vs. 85%**
Logistic Regression (BoW)	95.82%	85.71%	PASS +0.71%
Feed-Forward NN (Embedding)	97.40%	86.50%	PASS +1.50%
CNN	97.34%	85.57%	PASS +0.57%
CNN (No Stopwords)	97.09%	84.80%	FAIL −0.20%

All three primary models exceed the 85% accuracy target.

5 Analysis

5.1 Model Performance Hierarchy

Test accuracy ranking: **FFNN (86.50%)** > **LR (85.71%)** > **CNN (85.57%)** > **CNN-NS (84.80%)**.

The headline finding is that the FFNN outperforms the CNN. This is counter-intuitive given the CNN's richer inductive bias, and is examined in Section 5.2.

5.2 FFNN vs. CNN — Why the Simpler Model Wins Here

The FFNN has **4.16 million parameters** versus the CNN's **951,701** — roughly 4× more capacity. With 25,000 training examples, the FFNN's Embedding + Flatten path learns global word co-occurrence patterns across the entire 500-token sequence simultaneously.

Training curve analysis (Figures 1 and 2):

Both models overfit significantly. The FFNN exhibits more extreme overfitting:

- Training accuracy climbs to ≈99.99% by epoch 10.
- Validation accuracy plateaus at ≈85–86%.
- Training loss collapses to 0.0004 while `val_loss` rises to 0.9988 (a 250× divergence).

The CNN overfits less severely (train 97.34% vs. test 85.57%, an 11.77% gap), and its validation loss curve shows divergence from training loss starting around epoch 2–3.

Why CNNs typically dominate NLP: Conv1D's advantage becomes pronounced with (1) longer sequences, (2) pre-trained embeddings (GloVe, BERT), or (3) scarce training data where parameter efficiency matters. For this specific dataset (25k samples, vocabulary 5k, sequences padded to 500), the FFNN's global token-position attention provides sufficient discriminative signal.

Recommended fix: Adding `Dropout(0.3--0.5)` after `Dense(250)`, or applying early stopping at epoch 5–7 based on validation loss, would reduce overfitting and likely close the performance gap between models.

5.3 Logistic Regression as a Strong Baseline

LR achieves **85.71% test accuracy** — only 0.79% behind the FFNN and 0.14% ahead of the CNN. This demonstrates that the IMDB task has strong lexical signal: explicit sentiment vocabulary ("excellent", "terrible", "masterpiece", "boring") that a simple BoW representation can exploit without any learned representations.

The 10.11% train–test gap (95.82% vs. 85.71%) reflects overfitting despite sklearn's default L2 regularisation (`C=1.0`). Reducing `C` (e.g., `C=0.1`) would likely close this gap.

5.4 Effect of Stopword Removal (Task 6)

Removing NLTK English stopwords reduces CNN test accuracy from **85.57% to 84.80%**, a drop of **−0.77 percentage points**.

The Negation Story

NLTK's English stopword list includes sentiment-critical negation words:

> *no, not, nor, couldn't, didn't, doesn't, hadn't, hasn't, haven't, isn't, mustn't, needn't, shouldn't, wasn't, weren't, won't, wouldn't* (17 of 20 common negation words; *never*, *neither*, and *cannot* are not in NLTK's stopword list)

These are among the most powerful sentiment modifiers in natural language.

Concrete Before/After Decode Example

Table 2 shows a training review (review #5, Negative label) before and after stopword removal. Note how function words ("than", "it", "that", "the", "all", "other") are stripped alongside the structurally important "it's" and "those".

Table 2: Stopword removal: before and after decode of a sample review (#5)

Stage	First 20 decoded words
Before	`begins better than it ends funny that the russian <UNK> crew <UNK> all other actors it's like those scenes`
After	`begins better ends funny russian <UNK> crew <UNK> actors like scenes documentary shots br br spoiler part message <UNK>`

When "not" precedes a positive phrase (*"not a great film"* → *"great film"*) or "never" precedes a negative one (*"never disappoints"* → *"disappoints"*), the sentiment inverts. The CNN's 7-gram Conv1D filters learn these negation n-gram patterns as fixed-window features. Removing the negation token disrupts the n-gram and causes the filter to produce the wrong activation.

The relatively small drop (−0.77%) rather than a catastrophic collapse suggests:

1. Many reviews use explicit sentiment vocabulary that does not rely on negation.
2. The CNN partially recovers from surrounding context.
3. Neutral stopwords ("the", "a", "is") dominate the ≈111 removed words per review, diluting the impact of the 17 negation words.

6 Training Curves

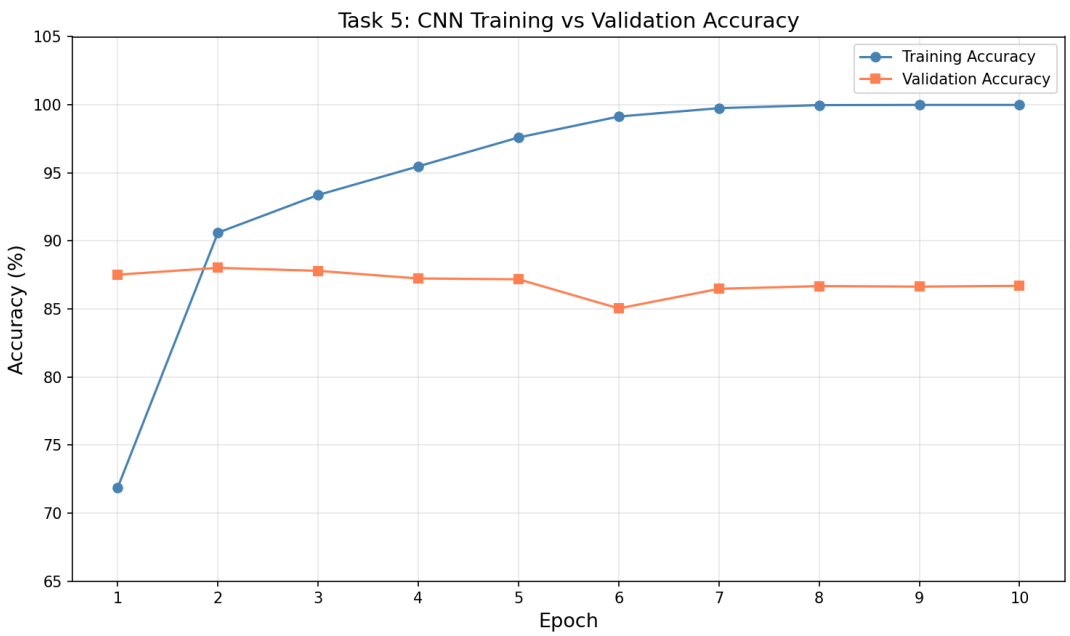

Figure 1: CNN training vs. validation accuracy per epoch. Both metrics are shown in percentage (%). The gap between training (≈97%) and validation (≈85%) accuracy at epoch 10 indicates significant overfitting.

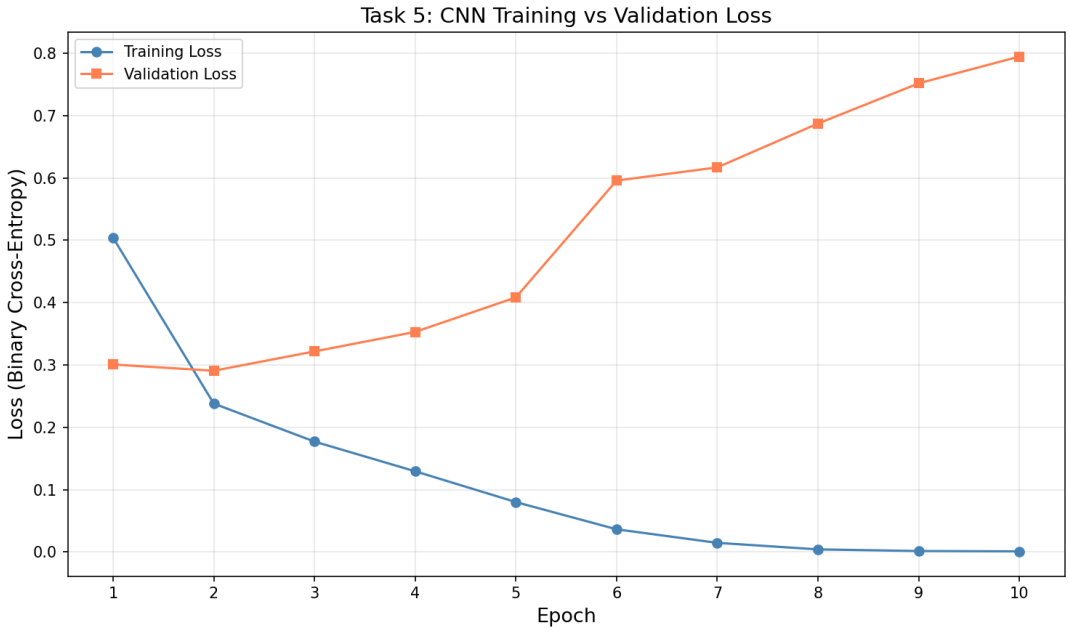

Figure 2: CNN training vs. validation loss (binary cross-entropy) per epoch. Validation loss begins rising after epoch 2–3 while training loss continues to drop — a classic overfitting signature.

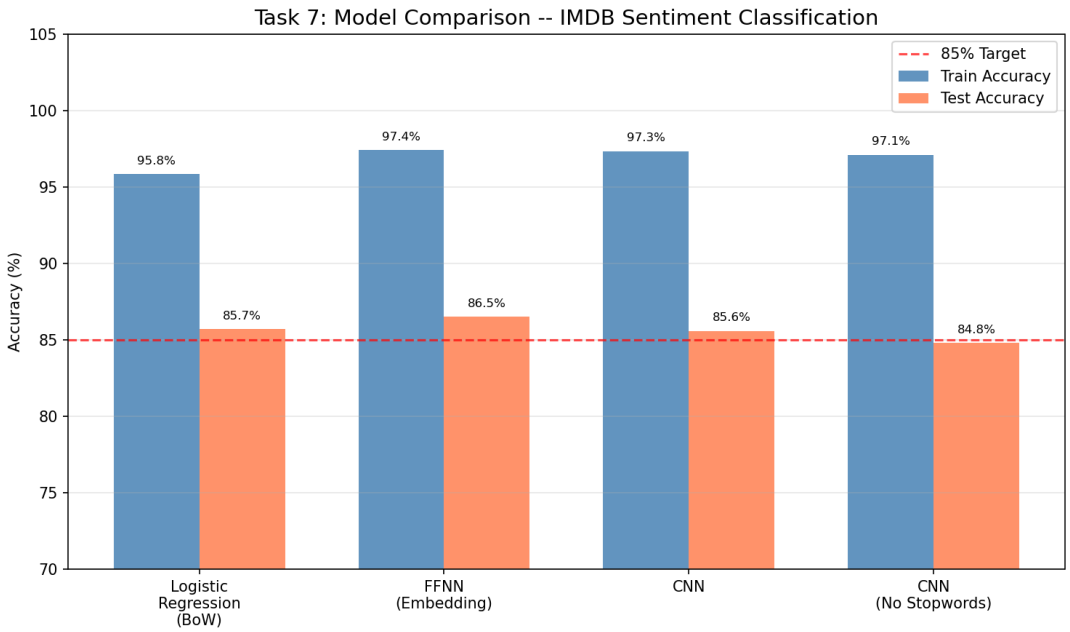

Figure 3: Model comparison: grouped bar chart of training and test accuracy for all four models. The horizontal dashed line marks the 85% target. Value labels show exact percentages. Note that all three primary models (LR, FFNN, CNN) clear the 85% threshold; the CNN without stopwords falls marginally short.

7 Conclusion

All three primary models exceed the 85% accuracy target on the IMDB test set: LR 85.71%, FFNN 86.50%, CNN 85.57%. Key conclusions:

1. **Bag-of-words LR is a competitive baseline** for sentiment classification — it actually outperforms the CNN by 0.14 percentage points (85.71% vs. 85.57%), because the IMDB task has strong lexical signal that a linear model can exploit directly.
2. **The FFNN outperforms the CNN** due to its 4× larger parameter count and sufficient training data for global pattern learning. Both models overfit heavily; Dropout or early stopping are the recommended remedies.
3. **Stopword removal hurts CNN accuracy (−0.77%)** because NLTK stopwords include 17 negation words that are critical for sentiment n-gram patterns (e.g., *"not good"*, *"never disappoints"*).

References

[1] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., and Potts, C. (2011). *Learning word vectors for sentiment analysis.* Proceedings of ACL 2011, pp. 142–150.

[2] Chollet, F. (2018). *Deep Learning with Python.* Manning Publications.

[3] Keras documentation. https://keras.io/api/layers/

[4] NLTK stopwords corpus. https://www.nltk.org/

Code: `Assignment 2/code/assignment2.py` *Output plots:* `Assignment 2/output/`